

Memi 2.7: Evidence-Scoped Design Intelligence

A Longitudinal Engineering Audit and Paired-Workflow Evaluation

Sarvesh Chidambaram

Memi Design, United States

ORCID: not supplied | Correspondence through [memi-design/memi](mailto:memi-design@memi.com)

Research artifact version 1.0 | 30 July 2026 | Memi 2.7.0

Abstract

Coding-agent skill systems promise lower context cost and better repository decisions, yet most public claims are not based on revision-matched tasks, immutable traces, or independent product verification. We report the longitudinal engineering history and preliminary evaluation of Memi 2.7, a repository-specific design-intelligence and skill-routing layer for coding agents. The audit traces failure modes from versions 2.0 through 2.7, including invalid tool schemas, non-blocking quality gates, optimistic treatment of unassessed evidence, release-surface drift, unsafe no-write behavior, and routing strategies that increased context on relevant but poorly bounded tasks. In a six-repository read-only paired pilot, Memi reduced counted tokens by 19.92% on average (95% bootstrap interval -0.23% to 40.07%) with wins on 4/6 repositories. In five multi-minute writable workflows, mean token savings were 0.55% (95% interval -51.23% to 37.30%), with four wins and one large regression. A single post-hoc Example 2 router calibration reduced tokens by 14.15% and wall time by 25.08%, but is not treated as confirmatory evidence. Engineering release checks passed 2154 tests across 300 files and nine OS/runtime combinations. The evidence supports publication of the software artifact, but not a universal 25% efficiency claim, cross-harness performance superiority, or senior-practitioner design-quality certification.

Keywords: coding agents; skill routing; design engineering; agent harness; paired evaluation; trace provenance; human-agent collaboration

1 Introduction

General-purpose coding agents must infer local architecture, conventions, design-system rules, and validation requirements before editing a repository. Skill frameworks can reduce this uncertainty by supplying task-specific procedures, but irrelevant or excessive context can make the same agent slower and more expensive. The useful research question is therefore not whether a skill appears relevant. It is whether an evidence-bounded routing policy improves a matched task without lowering work-product quality.

Memi is an open-source design-intelligence layer distributed as a command-line tool, Model Context Protocol (MCP) server, Agent Skills package, GitHub Action, and companion Studio application. It fingerprints a repository, produces read-only design evidence, selects a bounded skill stack, and records privacy-safe execution receipts. The design follows the Agent Skills and MCP tool contracts [1, 8]; trace identifiers are compatible with W3C Trace Context [13].

This paper makes four contributions:

1. a longitudinal error audit explaining how Memi progressed from a capability bundle to an evidence-scoped harness;
2. a paired evaluation across six repositories and five multi-minute product workflows, including negative results;
3. a failure analysis of routing, tool calls, retries, quality scoring, and release promotion; and
4. a reproducible artifact that binds tables and figures to committed JSON evidence by SHA-256.

2 Research Questions and Claim Policy

We separate questions that earlier versions incorrectly collapsed:

RQ1: Engineering readiness.

Does the exact 2.7 artifact pass its tests, type checks, supply-chain checks, installation smoke tests, and platform matrix?

RQ2: Efficiency.

Under matched conditions, does repository-specific Memi context reduce total tokens or wall time?

RQ3: Mechanism.

When routing fails, can traces identify whether the cost came from injected context, discovery, verification, retries, or fixture error?

RQ4: Quality.

Does automated product acceptance establish senior-practitioner design quality?

RQ5: Comparative performance.

Does Memi outperform other public harnesses under equivalent tasks and environments?

RQ1 is a software-release gate. RQ2 and RQ3 are empirical evaluation questions. RQ4 requires blinded practitioner calibration. RQ5 requires running other harnesses, not reading their feature lists. This *claim firewall* allows the package to ship without promoting incomplete research findings.

3 System and Trace Model

The router selects no more than two non-redundant skills under an 8 KB resource budget. Eligibility uses repository fingerprints and bounded regular expressions over dependencies, files, imports, scripts, frameworks, languages, task actions, platforms, and intent polarity. A high-priority niche skill may be exclusive. Content-addressed fitness events bind the selected skill revision to a baseline run, Memi run, model, reasoning setting, repository revision, and result. The system may promote, observe,

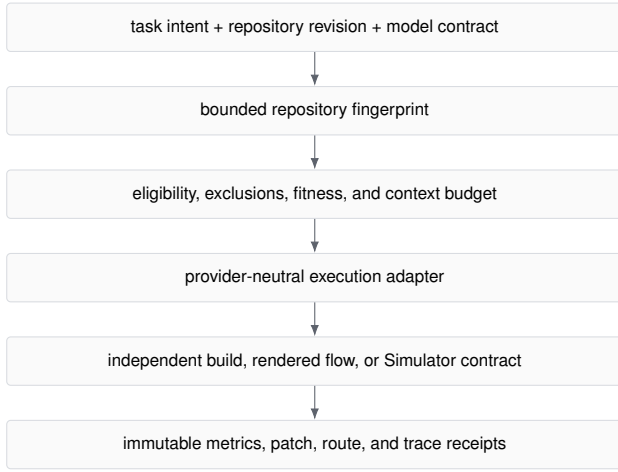


Figure 1: Memi 2.7 evaluation boundary. The model does not grade its own output; verification runs after the patch is captured.

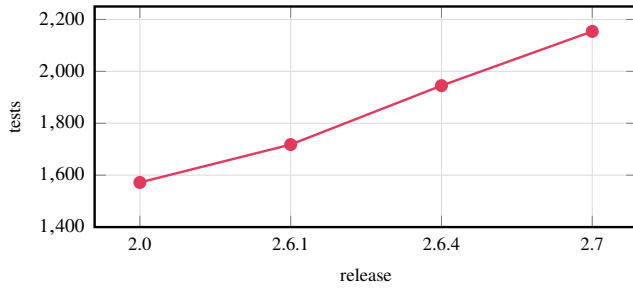


Figure 2: Documented test-count growth. Test count is engineering evidence, not a proxy for task quality.

quarantine, or abstain; negative evidence is not deleted.

Each run emits metadata-only events, tool-category counts, timing, token counts, verification status, and SHA-256 commitments. Raw prompts, hidden reasoning, absolute project paths, and secrets are excluded from durable public receipts. The model adapter fails closed when a provider reports a structured error even if its process exits with status zero.

4 Longitudinal Error Audit

The changelog was audited from v2.0.0 through v2.7.0 [3]. Table 1 lists errors that changed the research design. It is not a comprehensive feature history.

4.1 Test and package growth

The v2.0 release reported 1,572 tests across 205 files. The final 2.7 tree passed 2154 tests across 300 files. The published npm artifact contains 54 files and measures exactly 570048 bytes compressed (556.7 KiB) and 2019418 bytes unpacked (1.93 MiB). Production dependency auditing found no known vulnerabilities. These measurements answer only RQ1.

5 Evaluation Method

5.1 Paired conditions

For each experiment, baseline and Memi conditions used the same pinned repository revision, task, model, reasoning effort, fixtures, sandbox, and external verification contract. Personal skills, memory, plugins, and MCP servers were disabled. The Memi condition differed by receiving a deterministic repository preflight or routed skill context. Writable workflows ran in clean disposable clones. The patch was captured before verification to prevent test output from contaminating the measured change.

Total tokens are input, output, and reasoning tokens. For metric m , savings for pair i are

$$s_i(m) = \frac{B_i(m) - M_i(m)}{B_i(m)} \times 100, \quad (1)$$

where B_i is baseline and M_i is Memi. Positive values mean Memi used less. Arithmetic means describe an average task; medians describe the typical observed direction; weighted totals emphasize larger traces and are reported separately.

5.2 Uncertainty and preregistration

The committed reports use 10,000 nonparametric bootstrap samples [2]. The read-only study used seed 270; the writable workflow study used seed 270729. Wilson intervals describe win proportions [12]. The preregistered public claim required the lower 95% confidence bounds for token and latency savings to exceed 25%, with no quality regression. One pair per heterogeneous task is exploratory evidence, not a population estimate.

5.3 Quality measurement

Automated acceptance required real builds plus rendered browser flows or a pinned iOS Simulator journey. The historical runner stored passing acceptance as `qualityScore: 100`. Version 2.7 corrects the interpretation: automated acceptance can establish functional parity but is capped at 80. Scores above 80 require blinded senior-practitioner scoring and reliability evidence. Immutable historical records are not rewritten.

6 Results

6.1 Six-repository read-only pilot

Across 6 repositories, mean token savings were 19.92% (95% interval -0.23% to 40.07%); the interval crosses zero. Median savings were 24.25%, weighted savings were 24.25%, and 4/6 cases were positive. Mean latency savings were 0.50%. Example 3 was the only repository to clear 25% on tokens, latency, and tool calls. Example 4 regressed on all three metrics; Example 2 used more tokens despite completing faster. The study therefore rejected the universal 25% claim [6].

6.2 Multi-minute writable workflows

The five workflows lasted approximately 4.6 to 9.3 minutes per run. Mean token savings were 0.55% (95% interval -51.23% to

Table 1: Longitudinal defect-to-control audit. “Observed error” denotes a documented behavior, not a hypothetical risk.

Version	Observed error or invalid inference	Control introduced
2.0	The package bundled many surfaces but lacked a measured routing contract; feature presence was easy to confuse with agent benefit.	Separate install surfaces from task evidence and require repository-valid citations and verification commands.
2.1	Tailwind v4 color syntaxes were skipped; the codegen cache fingerprinted token count rather than token values; dotted MCP tool names were rejected by strict clients; the MCP version and root command were stale.	Shared color parsing, full-token fingerprints, protocol-valid tool names, structured errors, and executable MCP smoke tests.
2.1.1	Static MCP descriptions consumed about 26 KB per session.	Contract-preserving description reduction to about 13.5 KB, approximately 3,000 fewer context tokens before task work.
2.2	The quality gate emitted warnings without blocking writes; compliance guidance was prose without deterministic enforcement.	Severity-bearing findings, non-zero exits, blocked writes, and post-generation skill-compliance checks.
2.3	The CI gate required a critical severity the engine never emitted; JSON mode bypassed the gate; unassessed dimensions and fabricated screenshot evidence inflated scores.	Reachable severity gates, schema-v2 provenance, explicit not-assessed, removal of fabricated findings, and policy-hashed baselines.
2.3.1–2.4.1	History was appended twice; release binaries omitted optional native packages on Windows/Intel macOS; Marketplace metadata exceeded a hard length limit.	Route regression tests, clean packed installs, platform-matrix builds, and metadata contract tests.
2.6.1–2.6.2	Fresh installs exposed deprecated dependency chains; <code>-no-write</code> still wrote audit artifacts.	Dependency replacement, zero-finding production audit, and command-level no-write filesystem assertions.
2.6.3–2.6.4	Audit scores could outpace immutable proof; archive/network and shallow-clone boundaries were unsafe; provider tracing existed without full public release parity.	Fail-closed scorecards, source-bound release manifests, OIDC provenance, metadata-only runtime schemas, capability negotiation, and parity-specific gates.
2.7	Relevant skills could broaden discovery and verification; historical <code>qualityScore=100</code> was interpreted as professional quality; release promotion found Windows path, container-authentication, tag-permission, MCP namespace, and website drift errors.	Hard routing budgets, abstention, independent acceptance, an automated-quality ceiling, immutable incident evidence, and separate package, efficiency, certification, and parity decisions.

Table 2: Six-repository token and latency savings. Names are anonymized in the reader-facing report; immutable source records retain repository identities.

Repository	Tokens	Wall time
Example 1 (native iOS)	21.7%	−13.6%
Example 2 (mobile)	−3.7%	13.4%
Example 3 (multi-surface)	60.1%	28.2%
Example 4 (product UI)	−18.7%	−16.7%
Example 5 (web product)	33.3%	−14.3%
Example 6 (web landing)	26.8%	6.0%

37.30%), median savings were 19.43%, and weighted savings were 4.46%. Mean latency savings were 5.66% (95% interval −24.30% to 25.86%). Four of five tasks used fewer tokens, but Example 2 used 98.33% more. The wide intervals and negative case prevent a general claim [7].

Example 5 was the strongest stacked-skill result: 52.43% fewer tokens, 28.26% less wall time, and 44.12% fewer tool calls under the same accepted product contract. Example 1 provided bounded native evidence with 21.90% fewer tokens and 23.47% less wall time. Example 3 used 39.75% fewer uncached input tokens but doubled its tool calls. Relevant context was beneficial in some tasks and harmful in another; relevance alone is insufficient.

6.3 Router failure analysis

The canonical Example 2 run routed a React Native skill but broadened repository discovery and extra-platform verification. It used 98.33% more tokens and took 49.06% longer. Two subsequent narrower configurations still regressed. A third calibration made the task manifest and closest repository evidence authoritative, reduced injected context to 1,429 bytes, and forbade broad inventory unless a concrete failure justified expansion.

That single matched calibration used 14.15% fewer tokens, completed 25.08% faster, and reduced retries from 3 to 2. Tool-call “savings” were −57.89%, meaning Memi made more calls. The trace separated 19 baseline calls from 30 Memi calls: the difference occurred before the first edit through narrower search and read steps; Memi made fewer verification and repeated-verification calls afterward.

This result demonstrates a plausible mechanism, not a stable effect. The three router configurations are not homogeneous repeats and must not be averaged.

6.4 Robustness and sensitivity

The positive case count alone is weak evidence. Under an exact two-sided sign test with a null win probability of 0.5, the read-only pilot yields $p = 0.688$ and the writable workflows yield

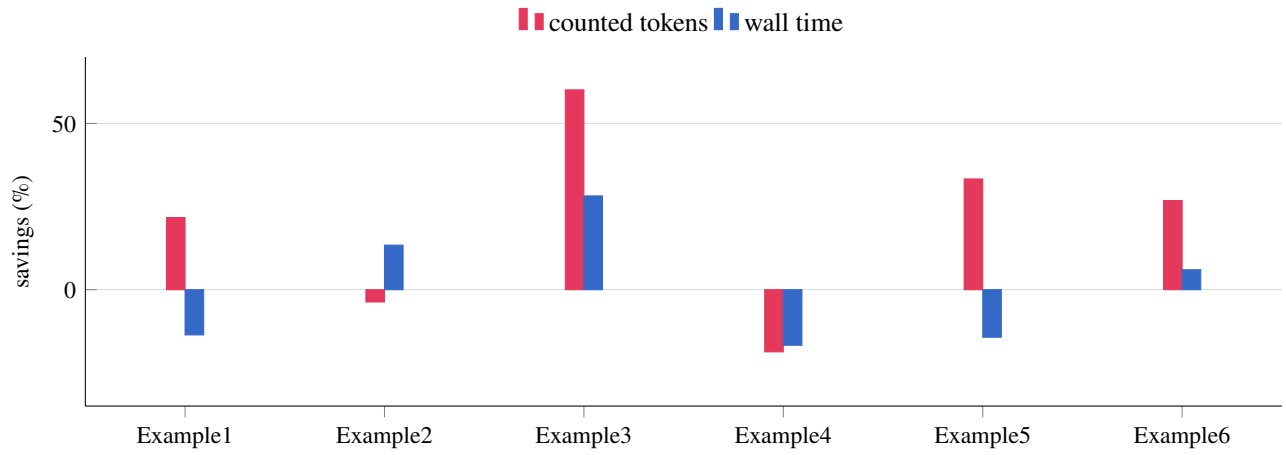


Figure 3: Revision-matched read-only pilot using presentation-safe identifiers. Positive values favor Memi. Negative results are retained.

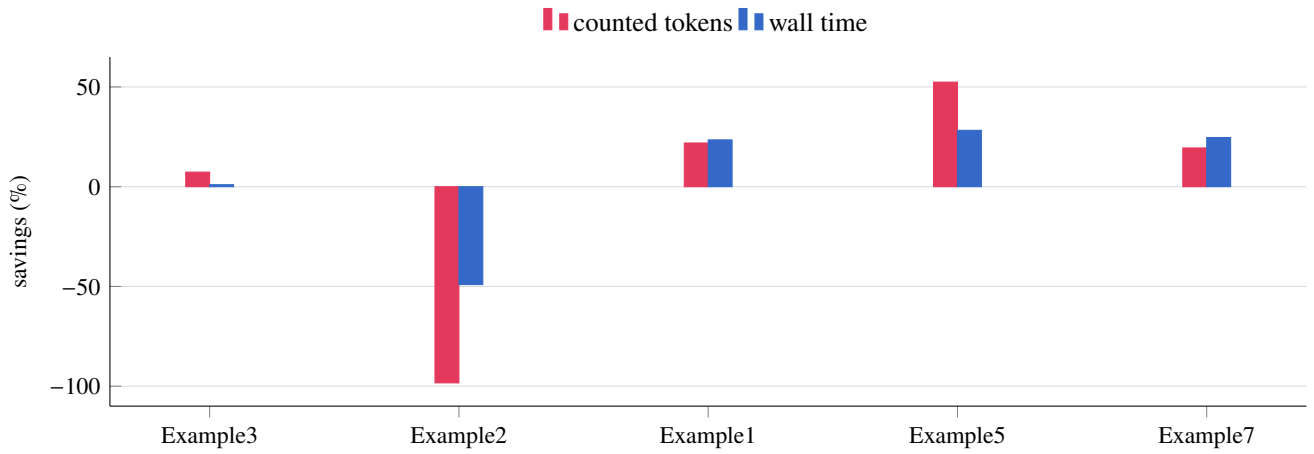


Figure 4: Canonical writable product workflows. Each condition passed its automated acceptance contract; Example 2 nevertheless regressed substantially.

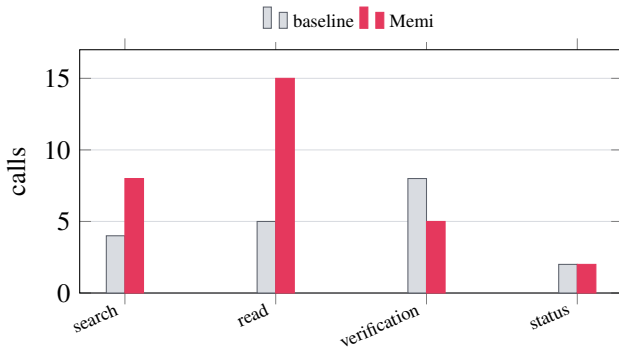


Figure 5: Privacy-safe Example 2 calibration tool profile. Call count is diagnostic, not a cost metric.

$p = 0.375$. Neither rejects the null. Leave-one-out mean token savings range from 11.88% to 27.64% in the six-repository pilot and from -12.42% to 25.27% in the writable study. The latter range changes sign, showing that the writable average is highly dependent on which example is omitted. These analyses strengthen the audit trail but do not add independent runs or increase the evidentiary grade.

7 Quality and Benchmark Readiness

All canonical workflow artifacts passed automated acceptance, but no blinded practitioner participated. DesignWorkBench v2 defines 15 disciplinary tracks and 300 task contracts, including product, graphic, motion, spatial/XR, agentic, iOS, cross-platform mobile, Android, native kits, systems, design engineering, service, content, interaction, and research. Only 60 fixtures are prepared; 0 are independently verified. 3 of 8 runners have receipts, and 0 practitioners are calibrated [4].

Consequently, professional certification is **BLOCKED**. A score of 100 in the historical workflow JSON means the automated contract passed. It does not mean the artifact matched a senior product designer, design engineer, motion designer, or native-platform specialist. New records identify the evidence source and cap automated acceptance at 80.

8 Comparison with Other Harnesses

Public documentation describes Superpowers as a composable, multi-harness software-development methodology [11]; ECC as a broad agent harness optimization system [9]; and Vercel's

Skills CLI as a cross-agent skill discovery and installation tool [10]. Table 3 compares documented capability, not speed or quality.

No baseline, Memi, Superpowers, and ECC runs were executed under one matched protocol. Skills CLI is a distribution surface rather than an execution condition. RQ5 is therefore **BLOCKED**. The paper reports no cross-harness performance rank. A preregistered multi-harness protocol is specified in the separate DesignWorkBench construction paper; its result cells remain empty until equivalent runs exist.

9 Release Execution and Incident Audit

Memi 2.7.0 was published from source commit 00be64b9 with npm signature, SLSA provenance, SBOM, four native binaries, per-asset checksums, a combined checksum manifest, an immutable container image, and a floating v2 Action tag [5]. The full PR matrix passed Linux, macOS, and Windows on Node 20, 22, and 24.

Release execution exposed four additional defects:

1. Windows binary startup duplicated the drive prefix (D:\D:\...); URL path conversion was replaced with `fileURLToPath` and tag-scoped repair tests.
2. The promotion workflow inspected private GHCR content before authentication; login now precedes immutable-image verification.
3. A GitHub App token could not move v2 across workflow-file changes; the exact source SHA was verified before authenticated Git moved the alias.
4. MCP publication returned 403 because the artifact declares `io.github.sarveshsea/memi` while the publisher is authorized for `io.github.memi-design/*`. The mismatch remains visible instead of rewriting immutable 2.7.0 package bytes.

The engine is published; cross-surface parity remains false. This distinction prevents an external promotion failure from erasing valid package evidence, while preventing npm success from being advertised as universal parity.

10 Threats to Validity

Sample size and heterogeneity. Each canonical task has one matched pair. Bootstrap intervals quantify observed variability but cannot compensate for limited coverage or repository selection.

Post-hoc calibration. Router v2 was informed by the Example 2 failure. Its positive result requires prospective homogeneous repeats.

Cost proxy. Subscription traces exposed tokens but not defensible billed USD. “Cheaper” means fewer counted tokens, not a dollar invoice.

Quality construct. Automated builds and rendered flows test functionality and selected accessibility behavior, not holistic design craft. Practitioner calibration is absent.

Provider coverage. The canonical runs use Codex CLI 0.145.0, gpt-5.6-sol, and medium reasoning. Claude Code returned a structured authentication failure under a revoked

credential, so no second-provider performance result is claimed.

Researcher involvement. The system author designed parts of the harness and analyzed the results. Immutable negative traces and predeclared gates reduce, but do not eliminate, researcher bias.

11 Reproducibility and Data Availability

The paper source, generator, summary JSON, benchmark summaries, readiness artifact, and release manifest are versioned in the public repository. Raw provider streams and patches remain outside the public tree because they may contain proprietary source or machine paths. Public summaries retain pinned revisions, run identifiers, model configuration, fixture contracts, aggregate methods, and content hashes.

Rebuild from the repository root:

```
python3 docs/research/memi-2.7-release-evidence/
generate_paper_assets.py
python3 ../latex-compile/scripts/compile_latex.py
docs/research/memi-2.7-release-evidence/main.tex
```

The PDF is a research artifact, not a peer-reviewed publication. The ORCID field remains explicitly “not supplied” because no identifier was present in the repository or supplied by the author at build time.

12 Conclusion

Memi 2.7 is a materially stronger engineering artifact than its earlier versions because its claims can now fail independently. The package passed its release gates; repository-specific context reduced tokens on most observed tasks; and traces explained both wins and regressions. The evidence does not support universal 25% savings, cross-harness superiority, or senior-level design certification. The next research milestone is not a larger marketing number. It is repeated, prospective, provider-diverse evaluation with blinded practitioner scoring and externally verified fixtures.

References

- [1] Agent Skills. Agent skills specification. <https://agentskills.io/specification>, 2026. Accessed 29 July 2026.
- [2] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1994. doi: 10.1201/9780429246593.
- [3] Memi Design. Memi changelog. <https://github.com/memi-design/memi/blob/main/CHANGELOG.md>, 2026. Repository artifact, accessed 30 July 2026.
- [4] Memi Design. Designworkbench v2 benchmark readiness artifact. <https://github.com/memi-design/memi/blob/main/docs/audits/memi-designworkbench-v2-readiness.json>, 2026. Machine-readable benchmark readiness evidence.

Table 3: Documentation-scoped capability comparison as inspected 29 July 2026. “Not documented” does not prove absence.

Capability	Memi 2.7	Superpowers	ECC	Skills CLI
Repository fingerprint routing	Built in	Not documented	Profile and context controls	Distribution, not routing
Hard context budget and abstention	Built in	Composable activation	Context-management guidance	Not documented
Content-addressed per-skill fitness	Built in	Eval framework documented	Learning/eval surfaces documented	Not documented
Paired cost/latency trace receipts	Built in	Skill behavior evals documented	Harness/eval tooling documented	Not documented
Design-specific professional benchmark	15-track foundation, not calibrated	Not documented	Not documented	Not documented
Cross-agent distribution	Skills, MCP, Action, plugins	Broad plugin support	Broad harness support	Core install surface

Table 4: Public release-surface status on 30 July 2026.

Surface	Status	Evidence
npm + provenance	PASS	2.7.0 latest; signature, attestation, SBOM
GitHub release	PASS	tag, release, binaries, checksums
Action + container	PASS	v2 source pin; GHCR image
Studio assets	PASS	pinned 2.5.0 artifacts verified
MCP Registry	BLOCKED	publisher namespace mismatch
Website parity	PARTIAL	stale docs; release endpoint returned 404

- [5] Memi Design. Memi 2.7.0 release. <https://github.com/memi-design/memi/releases/tag/v2.7.0>, 2026. Source tag and release assets, published 30 July 2026.
- [6] Memi Design. Memi 2.7 six-repository efficiency study. <https://github.com/memi-design/memi/tree/main/docs/case-studies/memi-2.7-six-repo>, 2026. Revision-matched paired pilot and machine-readable results.
- [7] Memi Design. Memi 2.7 routed workflow proof. <https://github.com/memi-design/memi/tree/main/docs/case-studies/memi-2.7-workflow-proof>, 2026. Pinned multi-minute workflow pairs and trace analysis.

- [8] Model Context Protocol. Server tools specification. <https://modelcontextprotocol.io/specification/2025-11-25/server/tools>, 2025. Protocol revision 25 November 2025.
- [9] Affaan Mustafeez and contributors. Everything claude code: Agent harness performance optimization. <https://github.com/affaan-m/ECC>, 2026. Accessed 29 July 2026.
- [10] Vercel Labs. Skills: The open agent skills tool. <https://github.com/vercel-labs/skills>, 2026. Accessed 29 July 2026.
- [11] Jesse Vincent and contributors. Superpowers: An agentic skills framework and software development methodology. <https://github.com/obra/superpowers>, 2026. Accessed 29 July 2026.
- [12] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.
- [13] World Wide Web Consortium. Trace context. <https://www.w3.org/TR/trace-context/>, 2021. W3C Recommendation.

A Evidence Commitments

Artifact	SHA-256
docs/case-studies/memi-2.7-six-repo/results.json	3329eec69ceeb22d68be6cec0172c84cbd55a63cc033bcf71a32c707b0104fd0
docs/case-studies/memi-2.7-workflow-proof/results.json	6f63f8ff8ee05ec278f95c0cb3fe25429cad4b29e50f70e140ca51e06e3e521c
docs/case-studies/memi-2.7-workflow-proof/tool-call-analysis.json	21274422e6d76bd2a21e349915ec44e51648f204c506e04d70bbcd85546da0bc
docs/audits/memi-designworkbench-v2-readiness.json	2c6cdfde67405664d32e4838ee5146de023e6e57df0abc77a9ddc1dc31e2939e
release-manifest.json	4e049b9de7a673e8c300312ee20b0256eb97ce8857dd2aba9acb3b095de5f2bd
CHANGELOG.md	67a9e8ef3b1b0b17a26a428f32b484b318bbfee5956ccc5e583df1858babdd98

B Release Decision Matrix

Claim	Decision	Required evidence
Memi 2.7 software artifact	Supported	Passed engineering, supply-chain, installation, and platform evidence
Mean savings on observed six-repo pilot	Descriptive only	Existing paired summary and interval
Universal savings above 25%	Rejected by current evidence	Lower 95% bounds above 25% for cost proxy and latency
Senior-practitioner quality	Unassessed	Blinded calibrated practitioners and verified private/holdout fixtures
Cross-harness superiority	Unassessed	Equivalent revisions, tasks, models, effort, fixtures, and acceptance contracts
USD cost savings	Unassessed	Provider billing evidence for every included pair

C Per-Workflow Metrics

Case	Wall	Tokens	Uncached	Tools	Routed skill(s)
Example 3	1.00%	7.31%	39.75%	-105.88%	motion-performance
Example 2	-49.06%	-98.33%	-40.67%	-25.00%	react-native-gen
Example 1	23.47%	21.90%	37.58%	-25.00%	swiftui-design-engineering
Example 5	28.26%	52.43%	26.98%	44.12%	emil-design-eng, interaction-design
Example 7	24.65%	19.43%	20.68%	33.33%	emil-design-eng, interaction-design